



ARQUITETURA DE SISTEMAS

AULA 9 - EVENT DRIVEN (EDA)



AGENDA DA AULA

- ❑ Introdução à Arquitetura Orientada a Eventos: Conceito básico e motivação para arquiteturas dirigidas por eventos.
- ❑ Topologias de Eventos (Broker vs. Mediator): Visão geral das duas principais abordagens e quando usar cada uma.
- ❑ Mecanismos e Comportamentos: Como funcionam as topologias Broker e Mediator internamente.
- ❑ Casos de Uso e Exemplos: Exemplos práticos de aplicação de cada topologia (fluxos de processamento de eventos).
- ❑ Padrões Relacionados: Workflow por eventos, Broadcast (pub/sub), Request-Reply e técnicas de prevenção de perda de dados.
- ❑ Trade-offs e Conclusões: Vantagens, desvantagens e critérios de escolha entre Broker e Mediator.



CONCEITOS

- ❑ Desacoplamento por Eventos: A EDA é um estilo arquitetural assíncrono e desacoplado em que componentes se comunicam por meio de eventos, ao invés de chamadas diretas. Cada componente reage a eventos de forma independente, aumentando a flexibilidade.
- ❑ Produção e Consumo de Eventos: Produtores de eventos geram eventos em resposta a ações ou mudanças de estado, enquanto consumidores de eventos escutam e processam esses eventos de forma assíncrona. Produtores não conhecem os consumidores, e vice-versa, garantindo baixo acoplamento.
- ❑ Comunicação Assíncrona: Diferente do modelo síncrono (request/response), na EDA os emissores disparam eventos e não esperam resposta imediata (padrão fire-and-forget). Os eventos transitam por um canal (ex.: fila ou tópico de mensagens) e podem ser processados em paralelo, melhorando escalabilidade.
- ❑ Escalabilidade e Reatividade: Como os componentes são independentes e a comunicação é orientada a mensagens, sistemas orientados a eventos tendem a escalar facilmente (horizontalmente) e a responder em tempo real a grandes volumes de eventos, adequado para aplicações distribuídas e com alto throughput.



MODELO DE REQUISIÇÃO VS. MODELO ORIENTADO A EVENTOS

- ❑ Arquitetura Tradicional (Request-Response): No modelo baseado em requisição, um cliente faz uma solicitação a um orquestrador (por exemplo, um controlador MVC ou API Gateway) que chama serviços sincronamente. Exemplo: um usuário solicita seu histórico de pedidos e o sistema executa chamadas determinísticas a serviços e banco de dados, retornando os dados ao final.
- ❑ Arquitetura Orientada a Eventos: Em contraste, no modelo orientado a eventos não há uma sequência fixa de chamadas síncronas. O sistema reage a acontecimentos: quando algo acontece (evento), esse fato é publicado e disparado para quem interessar. Cada serviço trata o evento de forma autônoma. Exemplo: em um leilão online, um evento "novo lance ofertado" é emitido quando alguém dá um lance, e diversos componentes podem reagir (atualizar o melhor lance, notificar usuários etc.), sem um componente central chamando todos.



MODELO DE REQUISIÇÃO VS. MODELO ORIENTADO A EVENTOS

- ❑ Diferenças de Temporização: No modelo de requisição, a execução é síncrona e sequencial, o solicitante espera a resposta de cada passo. Já em eventos, o processamento é assíncrono, ocorrendo possivelmente em paralelo e em tempos diferentes. O sistema lida bem com atrasos ou processamento offline de partes da tarefa.
- ❑ Acoplamento e Flexibilidade: Arquiteturas de requisição tendem a ser mais acopladas (componentes conhecem destinos e fluxos). Na EDA, os componentes são desacoplados: um serviço emite um evento sem saber quem irá consumi-lo. Essa flexibilidade facilita adicionar ou modificar consumidores sem impactar o produtor, embora traga desafios de visibilidade do fluxo completo.



COMPONENTES BÁSICOS EM ARQUITETURAS DE EVENTOS

- ❑ Evento: Registro ou mensagem que sinaliza que algo aconteceu no sistema. Contém dados sobre a ocorrência (por exemplo, "PedidoRealizado" contendo detalhes do pedido). É a unidade básica de comunicação.
- ❑ Produtor de Evento (Producer): Componente ou serviço que emite um evento quando uma determinada ação ou condição ocorre. Por exemplo, um serviço de pedidos emite o evento "PedidoRealizado" após gravar um novo pedido no banco.
- ❑ Consumidor de Evento (Consumer): Componente que ouve e reage a eventos de interesse. Executa lógica de negócio quando recebe um evento. Por exemplo, um serviço de faturamento escuta "PedidoRealizado" para emitir fatura, e um serviço de estoque reduz o inventário.



COMPONENTES BÁSICOS EM ARQUITETURAS DE EVENTOS

- ❑ Canal de Eventos: Meio de transporte dos eventos entre produtores e consumidores. Pode ser uma fila (p2p) ou um tópico (pub/sub) em um Broker de Mensagens (como RabbitMQ, ActiveMQ, Apache Kafka). Garante a entrega assíncrona dos eventos.
- ❑ Broker de Mensagens: Serviço intermediário (message broker) responsável por receber, armazenar e distribuir eventos aos consumidores. Atua como um hub central na topologia Broker, gerenciando filas/tópicos. Ex.: RabbitMQ orquestra a entrega de mensagens entre produtores e consumidores.
- ❑ Mediator (Mediador de Eventos): Componente central na topologia Mediator que orquestra o fluxo de eventos. Recebe um evento inicial e, a partir dele, dispara eventos/comandos sequenciais para coordenar múltiplos passos em um workflow. Age como um “controlador” do processo distribuído.



TOPOLOGIAS DE ARQUITETURA DE EVENTOS

As arquiteturas orientadas a eventos geralmente adotam duas topologias principais: Broker e Mediator. Ambas utilizam eventos e comunicação assíncrona, mas diferem no controle de fluxo e orquestração.

- ❑ Topologia Broker: Também conhecida como topologia de eventos encadeados ou event bus, não possui um mediador central. Os eventos são transmitidos através de um broker e cada componente reage publicando novos eventos para dar sequência ao processamento. É orientada a choreography (coreografia), isto é, o fluxo emerge da interação dos componentes sem controle central.
- ❑ Topologia Mediator: Introduz um mediador central que coordena ativamente os passos de processamento do evento. O mediador recebe o evento inicial e decide quais passos (sub-eventos) executar, em qual ordem (sequencial ou paralelo), encaminhando comandos a componentes específicos. É uma abordagem de orchestration (orquestração), com controle global do workflow.

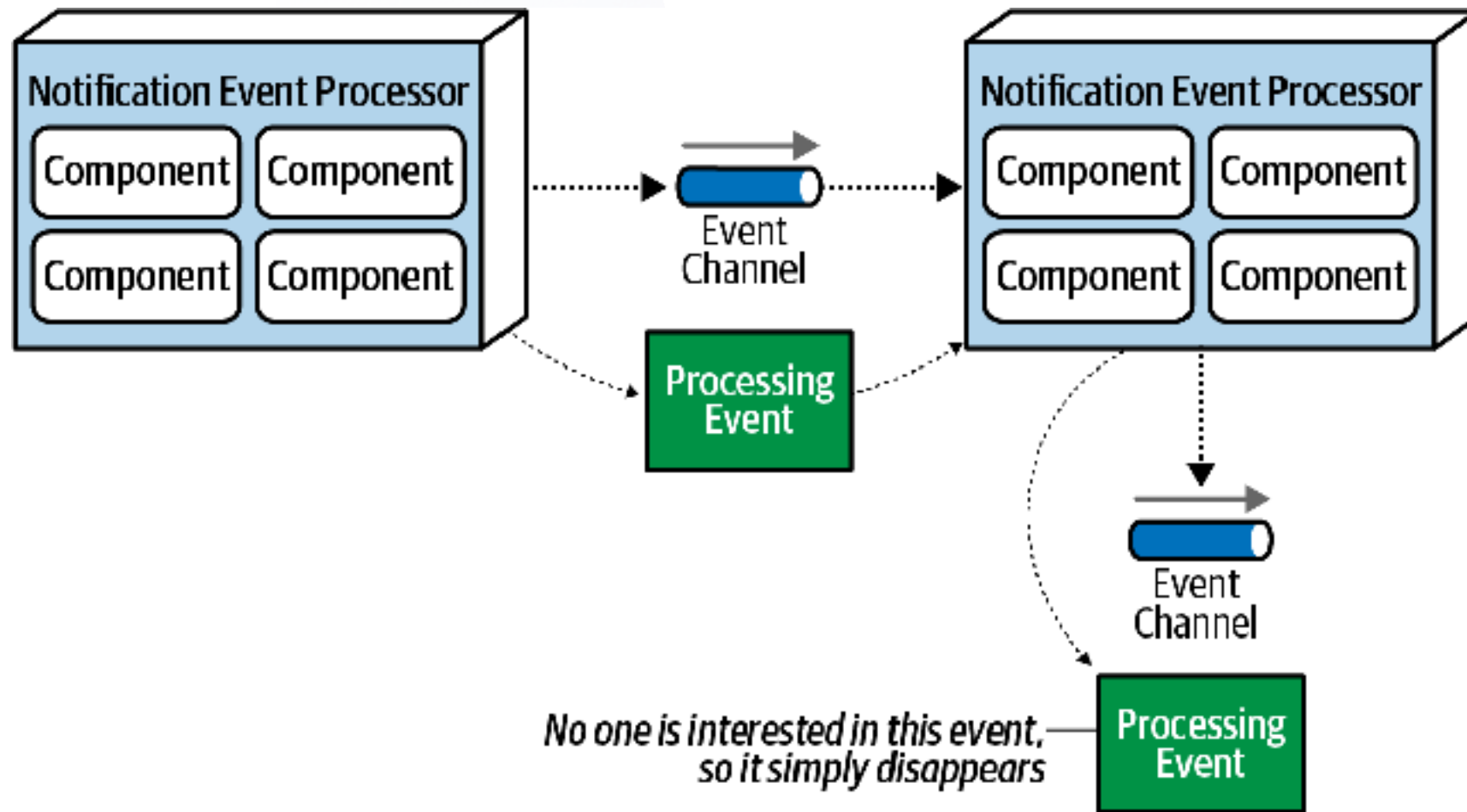


TOPOLOGIAS DE ARQUITETURA DE EVENTOS

- ❑ Quando Usar Broker: Adequada para fluxos simples ou altamente distribuídos onde não há dependências complexas entre passos. Favorece desempenho e escalabilidade, pois elimina coordenação central. Útil quando eventos podem ser processados independentemente e qualquer ordem de conclusão é aceitável.
- ❑ Quando Usar Mediator: Indicada para processos de negócio complexos que requerem múltiplas etapas coordenadas ou lógica condicional. Se o evento desencadeia várias ações que precisam acontecer em sequência definida (ou com lógica de decisão), o mediador garante ordem e tratamento de erros entre passos. Útil para manter consistência em transações distribuídas.



TOPOLOGIA BROKER



Exemplo de fluxo em topologia Broker, com eventos propagados em cadeia via um Event Broker (middleware de mensagens).

TOPOLOGIA BROKER

- ❑ Sem Orquestrador Central: No padrão Broker, o sistema não possui um controlador global de fluxo. Cada evento disparado é tratado de forma independente por quaisquer consumidores inscritos. Cada componente faz sua parte e, ao terminar, anuncia o que fez via novo evento, sem esperar ou saber quem tratará adiante.
- ❑ Encadeamento de Eventos: Um evento inicial (por exemplo, PedidoCriado) é publicado no broker. Um único processador consome esse evento e realiza sua função (ex.: serviço de pagamento processa o pagamento), então emite um evento de processamento (ex.: PagamentoEfetuado) no broker. Múltiplos consumidores podem estar escutando esse novo evento. Por exemplo, serviço de estoque e de notificações escutam PagamentoEfetuado e agem em paralelo.



TOPOLOGIA BROKER

- ❑ Broadcast Assíncrono (Pub/Sub): A comunicação geralmente segue o modelo publish/subscribe: eventos publicados no broker são um broadcast para todos os interessados. Os consumidores decidem ignorar ou processar. Essa difusão assíncrona maximiza o desacoplamento. O produtor não sabe quem consome, e os consumidores não competem entre si (todos recebem a mensagem). O broker frequentemente usa tópicos para organizar os eventos por tipo.
- ❑ Fluxo Dinâmico: Como não há sequência pré-definida imposta por um mediador, o fluxo de processamento pode se adaptar dinamicamente. Componentes podem ser adicionados ou removidos sem alterar os existentes: basta se inscreverem nos eventos relevantes. O sistema se comporta como uma coreografia, onde cada serviço reage conforme necessário. Isso traz agilidade e permite evolução fácil da arquitetura (novo serviço pode começar a escutar um evento e reagir, adicionando funcionalidade sem tocar nos outros).



VANTAGENS

- ❑ Alta Escalabilidade: Componentes são altamente desacoplados e comunicam via broker, podendo escalar independentemente. Como não existe ponto central de coordenação, o throughput do sistema aumenta linearmente com a adição de consumidores. A topologia Broker é conhecida por suportar grande volume de eventos com baixa latência.
- ❑ Desempenho e Responsividade: Por operar de forma assíncrona e paralela, essa arquitetura atinge alto desempenho. Eventos são processados de imediato pelos consumidores disponíveis, sem filas globais de espera. A ausência de um mediador eliminando etapas sequenciais faz com que o sistema responda rapidamente a novas ocorrências (responsiveness), adequado para aplicações em tempo real.
- ❑ Flexibilidade e Extensibilidade: Novos serviços podem ser adicionados ouvindo eventos existentes sem impactar os produtores. A arquitetura favorece evolução: para adicionar uma funcionalidade, basta adicionar um novo consumidor para algum evento. Não é necessário modificar o fluxo central (já que não há um). Isso reduz o custo de manutenção e permite evoluir o sistema de forma iterativa.



DESAFIOS

- ❑ Falta de Controle Global: Como contraponto, não haver um mediador significa ausência de orquestração central. Não há visibilidade global do workflow, cada componente conhece apenas sua parte. Isso dificulta entender ou garantir a ordem de processamento em fluxos multi-etapas. Se um processo de negócio requer etapas ordenadas estritamente, o Broker puro não fornece essa garantia por si só.
- ❑ Erro e Consistência: O gerenciamento de erros em Broker é complexo. Se um passo falha, não existe um componente central ciente para reagir ou reiniciar o fluxo. Pode ocorrer inconsistência de dados: por exemplo, se alguns serviços consumirem o evento inicial e outros falharem, o sistema fica em estado parcialmente atualizado. Não há mecanismo nativo de reversão ou retry global, essas medidas precisam ser planejadas (por exemplo, consumidores com lógica de compensação ou monitoramento externo).
- ❑ Transações Distribuídas Difíceis: Em virtude da natureza assíncrona, transações distribuídas (two-phase commit) ou garantia ACID de ponta a ponta são praticamente inviáveis na topologia Broker. Uma vez publicado um evento e processado por vários serviços, não há como “desfazer” facilmente se algo der errado. Isso exige adotar estratégias alternativas de consistência eventual ou a implementação de sagas (sequências de compensações) caso seja necessária confiabilidade transacional.



TOPOLOGIA MEDIATOR

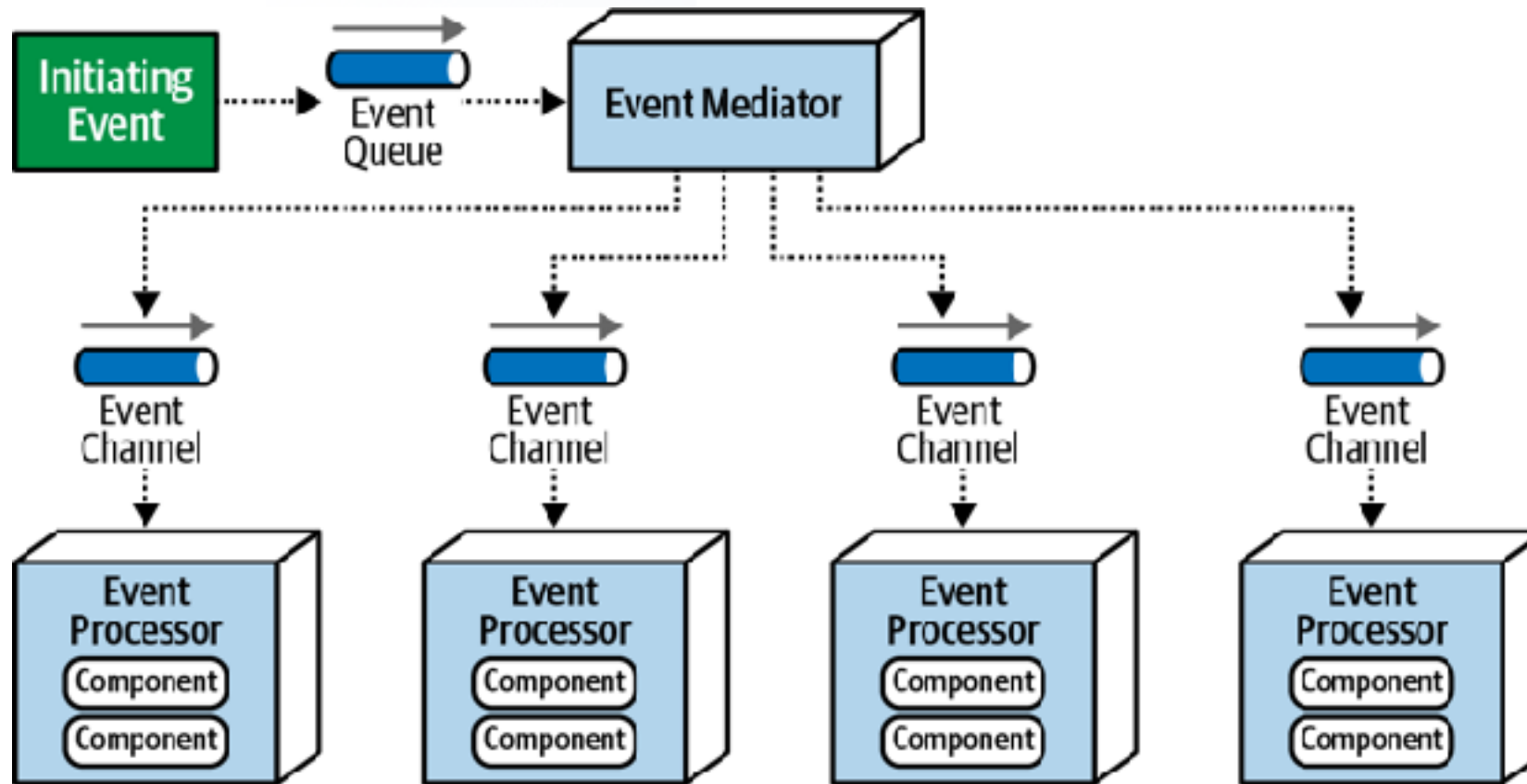


Diagrama da topologia Mediator, com um Event Mediator central orquestrando vários processadores via canais dedicados.

TOPOLOGIA MEDIATOR

- ❑ Orquestrador Central: Diferente da coreografia do Broker, o Mediator atua como um orquestrador global. Ele sabe todas as etapas necessárias para processar o evento inicial. Ao receber o evento, aciona os componentes na ordem correta, impondo uma lógica de workflow definida (por exemplo: 1º pagamento, 2º atualização de estoque, 3º envio de confirmação).
- ❑ Comunicação Ponto-a-Ponto: O mediador envia eventos como comandos direcionados. Em vez de publicar de forma broadcast em um tópico para qualquer um, ele coloca cada evento de processamento em uma fila específica para o serviço alvo. Essa entrega point-to-point significa que cada sub-evento tem um consumidor designado (ex.: um comando “ProcessarPagamento” vai apenas para o serviço de pagamento). Os consumidores, ao concluir sua tarefa, podem responder ao mediador (por exemplo, retornando um evento “PagamentoConcluido” em outra fila que o mediador escuta).



TOPOLOGIA MEDIATOR

- ❑ Sequenciamento e Controle de Fluxo: O Mediator decide sequência e paralelismo. Passos que podem ocorrer em paralelo, ele dispara simultaneamente; passos que dependem de outros, ele agenda após receber confirmação dos anteriores. Ele também pode aplicar lógica condicional: por exemplo, se PagamentoConcluido vier com resultado "negado", o mediador pode interromper o fluxo e emitir um evento de cancelamento ao invés de prosseguir ao envio.
- ❑ Manutenção de Estado: Uma vantagem chave é que o Mediator pode manter o estado contextual da transação enquanto coordena. Ele reúne informações dos passos, podendo agregá-las. Por exemplo, se um passo produz um dado necessário para outro, o mediador armazena essa informação temporariamente e repassa no próximo comando. Isso confere ao sistema uma visão consistente do workflow, semelhante a uma sessão transacional distribuída controlada pelo mediador.



VANTAGENS

- ❑ Controle de Workflow: A principal vantagem do Mediator é fornecer controle explícito sobre o fluxo de eventos. Isso permite implementar facilmente regras de negócio complexas, garantindo que cada passo ocorra na ordem correta. O mediador pode aplicar lógicas de decisão, repetir etapas ou bifurcar caminhos (workflows condicionais). Coisas difíceis de realizar somente com Broker.
- ❑ Tratamento de Erros Centralizado: Com um mediador, fica mais simples gerenciar falhas. Erro em uma etapa? O mediador detecta a ausência de confirmação ou recebe um evento de erro e pode tomar ações compensatórias (ex.: reverter passos já feitos via eventos de compensação) ou tentar novamente aquela etapa. Também é possível implementar lógica de timeout (se um serviço não responder em X tempo, o mediador aborta ou redireciona). Isso traz mais robustez e recuperabilidade ao processo.



VANTAGENS

- ❑ **Consistência de Dados:** O Mediator central mantém o contexto da transação, então é mais fácil assegurar consistência. Por exemplo, se todas as etapas dependem de um identificador único do pedido, o mediador repassa esse identificador em cada comando, evitando divergências. Como ele coordena confirmações, pode garantir que ou todas as etapas cruciais ocorreram ou, em caso de falha, nenhuma mudança parcial fique pendente sem tratamento.
- ❑ **Possibilidade de Reinício do Processo:** Em caso de falha catastrófica, o mediador sabe em qual etapa o processo parou e o estado atual. Assim, em alguns cenários pode ser possível implementar um recomeço/retry do fluxo do ponto de falha (coisa que o Broker puro não faz, pois nenhum componente isolado tem essa visão). Embora nem sempre trivial, pelo menos o mediador oferece um ponto único para implementar reinício ou retomada se necessário.



DESAFIOS

- ❑ **Maior Acoplamento:** Uma desvantagem é que o Mediator introduz acoplamento entre processos. Os event processors deixam de ser completamente independentes: eles estão agora ligados por uma sequência controlada. Alterar o workflow exige modificar o mediador (codificando novos passos ou lógica), criando dependência lógica central. Os componentes perdem um pouco da autonomia, já que não anunciam livremente seus eventos, mas sim executam comandos do mediador.
- ❑ **Escalabilidade e Desempenho Inferiores:** O mediador adiciona uma camada extra que pode se tornar um ponto de contenção. Em cargas muito altas ou fluxos muito complexos, o mediador pode ser sobrecarregado coordenando múltiplos eventos, limitando a escalabilidade comparado ao Broker puro. Mesmo podendo instanciar mediadores por domínio ou escalar o mediador horizontalmente, ainda assim há um overhead de orquestração que reduz a performance global (latência maior, pois muitas etapas ocorrem sequencialmente sob controle central).



DESAFIOS

- ❑ Ponto Único de Falha: Sem cuidados extras, o Mediator pode ser um single point of failure: se ele cair, processos em andamento ficam interrompidos. Mitigações incluem replicar mediadores por domínio (vários mediadores cada um cuidando de um conjunto de eventos) ou ter mediadores redundantes. Porém, isso complica o design e pode reintroduzir parte dos desafios (por exemplo, coordenar mediadores entre si para não duplicar processos).
- ❑ Complexidade na Implementação: Modelar workflows no mediador pode ser trabalhoso, especialmente se o fluxo tiver muitas variações ou condições excepcionais. Às vezes, uma lógica de negócio muito dinâmica ou com muitas exceções é difícil de codificar em um mediador de forma declarativa. Nesses casos, pode-se combinar abordagens (por exemplo, usar Broker para partes imprevisíveis e Mediator para a parte principal), aumentando também a complexidade geral do sistema.



BROKER VS. MEDIATOR

COMPARAÇÃO DIRETA



BROKER VS. MEDIATOR - COMPARAÇÃO DIRETA

- ❑ **Coordenação do Fluxo:** Broker opera via coreografia distribuída, sem controle central, o que é adequado para fluxos simples ou altamente paralelizáveis. Mediator utiliza orquestração centralizada, ideal para fluxos complexos e ordenados. Trade-off: coreografia oferece mais dinamismo e desacoplamento, orquestração oferece controle e previsibilidade.
- ❑ **Desempenho e Escalabilidade:** Broker tende a oferecer maior throughput e menor latência em grande escala, pois elimina a figura de um coordenador serializando passos. Mediator, embora escalável, tem overhead adicional e pode se tornar gargalo. Performance ligeiramente menor e necessidade de escalar também o mediador conforme aumenta a carga. Broker maximiza paralelismo, Mediator introduz etapas sequenciais quando necessário (impactando tempo de processamento total).
- ❑ **Tolerância a Falhas:** Na topologia Broker, cada componente falha isoladamente sem paralisar o fluxo geral (outros eventos continuam fluindo). Entretanto, a falha de um passo pode passar despercebida globalmente e deixar processamento incompleto. No Mediator, falhas podem ser detectadas e tratadas (maior confiabilidade transacional), porém o mediador em si é um componente crítico cuja falha impacta todo processo que ele gerencia. Broker oferece fault tolerance por descentralização, Mediator oferece recuperação estruturada ao custo de um ponto central sensível.



BROKER VS. MEDIATOR - COMPARAÇÃO DIRETA

- ❑ Acoplamento e Evolução: Broker tem baixo acoplamento. Fácil adicionar ou remover consumidores sem tocar nos demais; porém, difícil garantir integridade de fluxo complexo. Mediator tem acoplamento moderado: os componentes seguem um contrato de participação no fluxo orquestrado. Adicionar uma nova etapa exige alterar o mediador e possivelmente integrar com passos existentes. Em sistemas com Mediator, a evolução requer planejamento central do workflow; já no Broker, a evolução é mais emergente (mas precisa cuidado para não virar uma "bagunça" de eventos sem coordenação).
- ❑ Casos de Uso Típicos: Broker é comum em eventos de sistema, streams de dados, integração simples entre serviços. Por exemplo, sistemas de logging distribuído, notificações de sensores IoT, onde cada evento é tratado independentemente. Mediator aparece em sagas de negócio, orquestração entre microserviços (ex.: um processo de checkout que envolve pagamento, estoque, envio, etc.), ou em implementação de fluxos transacionais distribuídos que precisam de monitoramento central.



DESIGN PATTERNS



PADRÃO EVENT WORKFLOW

- ❑ Orquestração de Eventos: Refere-se a usar eventos encadeados para realizar um workflow de negócio completo. No padrão Mediator, isso se manifesta diretamente: o mediador implementa o workflow emitindo comandos e aguardando resultados. Já em Broker puro, workflows podem ser construídos via coreografia, mas sem garantia determinística de sequência: adequa-se apenas a fluxos onde a ordem não é crítica ou é implicitamente gerenciada.
- ❑ Event Choreography vs. Orchestration: Em cenários simples, pode-se usar coreografia de eventos (vários serviços reagindo a um evento de maneira distribuída) para compor um workflow. Ex: Ao criar um usuário novo, serviços de boas-vindas, criação de perfil padrão e envio de e-mail podem reagir em paralelo ao evento “UsuarioCriado”. Isso funciona sem mediador porque as ações são independentes e a ordem não importa. Entretanto, para fluxos com regras de negócio sequenciais ou condicional, a orquestração (Mediator) se torna necessária. Muitas arquiteturas combinam os dois padrões: usam workflows por eventos via Mediator para a sequência principal e permitem que sub-eventos sejam broadcast para reações adicionais menos críticas.

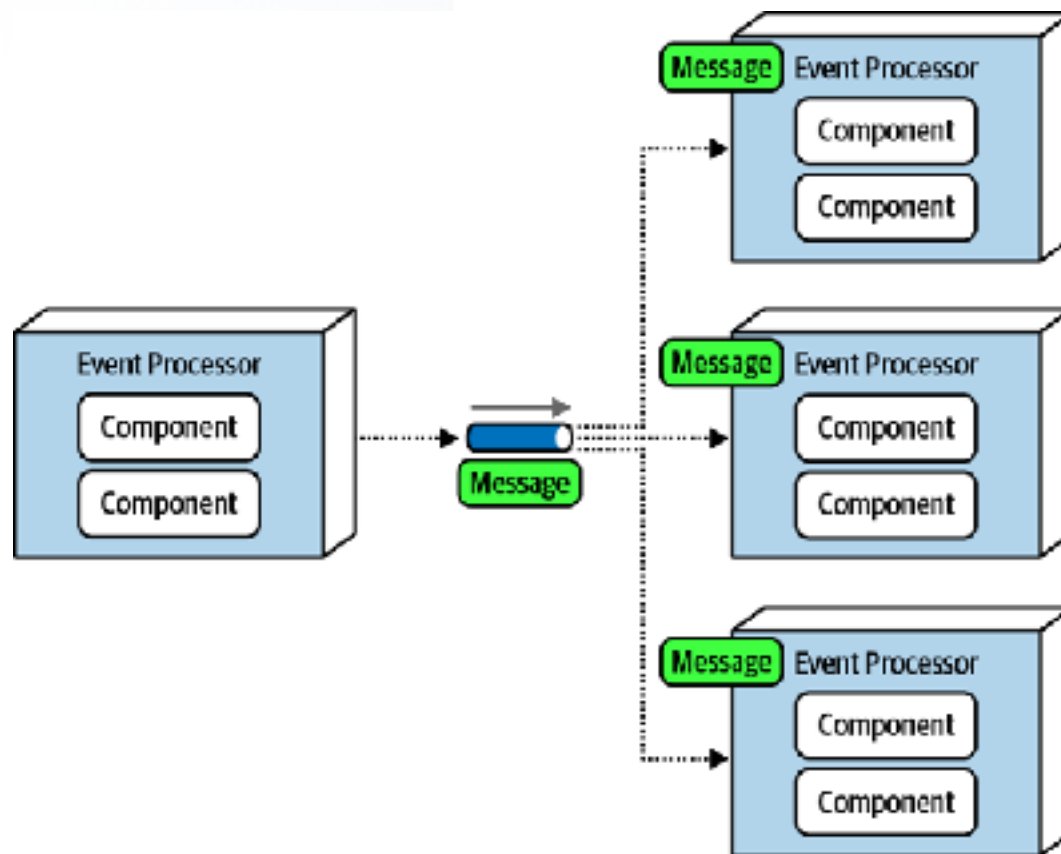


PADRÃO EVENT WORKFLOW

- ❑ Comunicação Síncrona x Workflow Assíncrono: É importante notar que o workflow por eventos mantém a natureza assíncrona: os componentes realizam tarefas sem bloquear uns aos outros, mas o encadeamento lógico é preservado. Isso contrasta com implementar o fluxo via chamadas síncronas (que poderiam travar um serviço aguardando outro). O padrão de orquestração por eventos busca o melhor dos dois mundos: coordenação de alto nível, mas com execução assíncrona distribuída.



PADRÃO BROADCAST (PUB/SUB)



Exemplo do padrão broadcast, onde um evento é distribuído a múltiplos processadores de evento simultaneamente (multi-cast).

PADRÃO BROADCAST (PUB/SUB)

- ❑ Broadcast de Eventos: Broadcast em EDA significa que um evento publicado pode ser recebido por múltiplos consumidores simultaneamente. Esse padrão baseia-se no modelo publish/subscribe (pub/sub): produtores publicam eventos em um tópico/canal, e todos os consumidores inscritos (subscribers) nesse tópico recebem a mensagem. É a forma fundamental de comunicação no Broker e também ocorre dentro do Mediator (quando este emite comandos, embora direcionados).
- ❑ Desacoplamento Máximo: O broadcast promove o mais alto nível de desacoplamento: o produtor não sabe quantos nem quais componentes receberão o evento. Cada consumidor decide independentemente o que fazer. Por exemplo, um serviço de preços emite evento "CotaçãoAtualizada" e diversos outros serviços (monitoramento, alertas, pedidos pendentes) recebem e atuam, sem que o emissor precise conhecer cada destino. Se nenhum consumidor estiver inscrito, o evento simplesmente não é consumido por ninguém. O produtor não precisa se preocupar, pois seu papel é só anunciar.



PADRÃO BROADCAST (PUB/SUB)

- ❑ Uso em Topologia Broker: No Broker, tipicamente utiliza-se broadcast para distribuir eventos de forma assíncrona e em paralelo. Exemplo: um evento NovoProduto é broadcast e serviços de catálogo, recomendações e estoque recebem ao mesmo tempo para atualizar seus registros locais. Isso torna o sistema reativo, propagando mudanças rapidamente a todas as partes interessadas. Implementações comuns incluem tópicos JMS, exchanges do AMQP ou mecanismos como Kafka (streams) onde todos consumidores podem ler todos eventos.
- ❑ Considerações de Broadcast: Embora poderoso, o broadcast significa que consumidores não coordenados podem levar a comportamentos indesejáveis. É crucial definir claramente o contrato do evento (o que ele representa) para que cada consumidor interprete corretamente. Também, se muitos consumidores para um evento executam operações pesadas simultaneamente, pode haver picos de carga. Em suma, broadcast oferece escalabilidade horizontal (vários consumidores), mas exige design cuidadoso para evitar sobreposições ou processamento redundante.

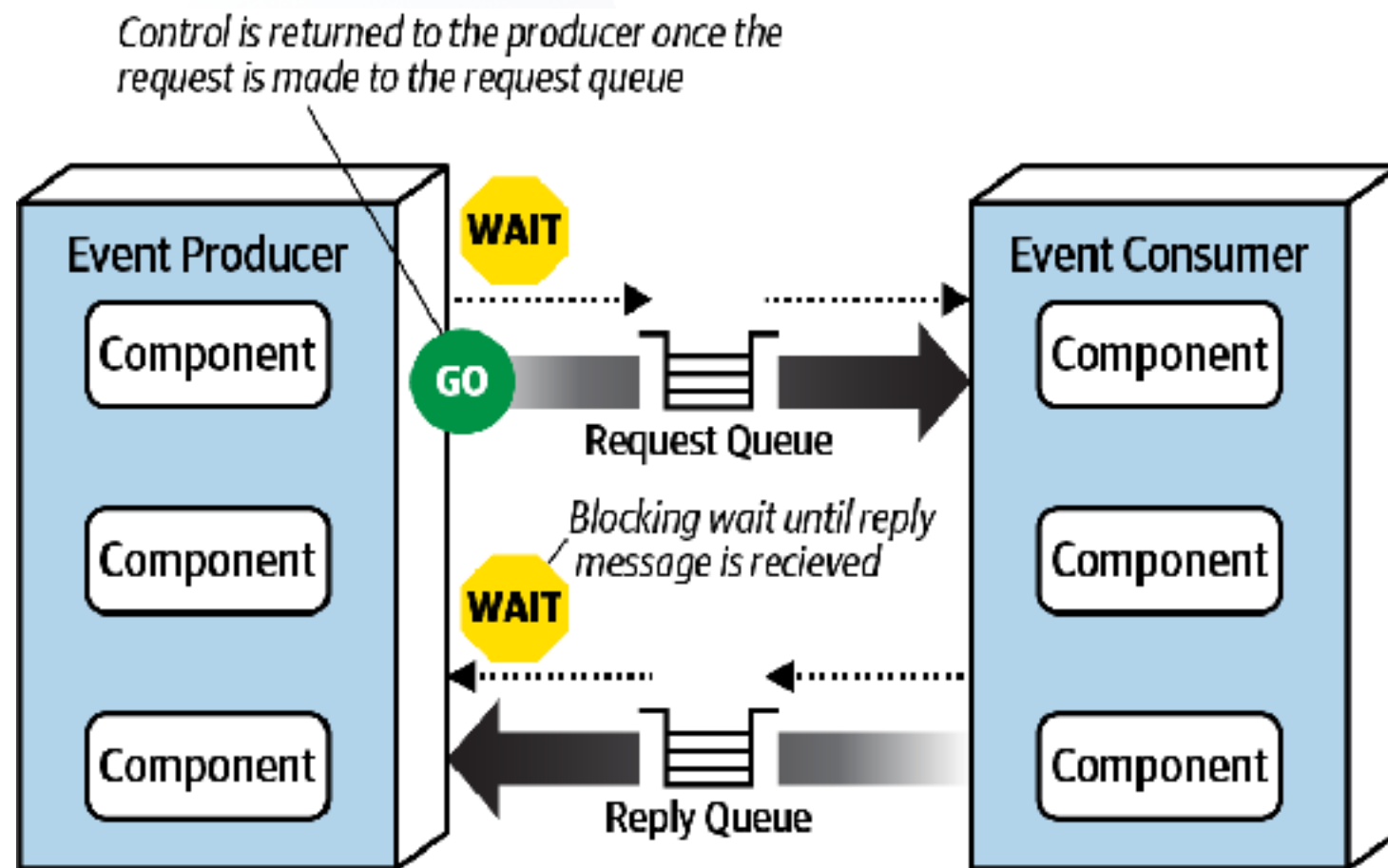


PADRÃO BROADCAST (PUB/SUB)

- ❑ Broadcast vs. Ponto-a-Ponto: Contrastando com comandos p2p (onde um evento tem destino único), o broadcast multiplica o impacto de um evento. Em arquiteturas reais, usa-se broadcast para eventos de domínio divulgativos ("algo ocorreu") e ponto-a-ponto para comandos direcionados ("faça algo"). Ambos coexistem: por exemplo, no Mediator, o mediador pode emitir um comando específico para um serviço (ponto-a-ponto) e também publicar um evento geral para outros ouvintes interessados (broadcast de notificação).



PADRÃO REQUEST-REPLY



Mecanismo request-reply: o produtor envia um evento de requisição (indicativo “GO”) e então faz espera bloqueante (“WAIT”) pela resposta em outra fila.

PADRÃO REQUEST-REPLY

- ❑ Necessidade de Resposta: Embora EDA seja naturalmente “fire-and-forget”, há casos em que um serviço solicitante precisa de uma resposta a um dado evento (por exemplo, pedir dados adicionais a outro serviço). O padrão Request-Reply em mensagens implementa uma comunicação pseudo-síncrona sobre infraestrutura assíncrona, permitindo obter uma resposta a uma requisição via eventos.
- ❑ Implementação com Correlation ID: Uma técnica comum é o uso de Correlation ID. O serviço A envia um evento de requisição (ex.: ConsultarSaldo) para uma fila ou tópico, incluindo nos cabeçalhos um ID único e indicando um canal de resposta (header reply-to). O serviço B consome o pedido, processa e emite um evento de resposta (ex.: SaldoAtual) endereçado ao canal de resposta, copiando o correlation ID. O serviço A escuta a resposta e, ao recebê-la, alinha pelo correlation ID para correlacionar à requisição original. Assim, completa-se o ciclo request-reply sem bloqueio: A ficou aguardando a mensagem de resposta em seu próprio ritmo (poderia estar fazendo outra coisa enquanto isso).



PADRÃO REQUEST-REPLY

- ❑ Uso de Filas Temporárias: Outra abordagem é usar fila temporária exclusiva para a resposta. O solicitante cria uma fila ad-hoc apenas para aquela requisição e informa esse endereço no evento de requisição. O serviço respondente envia de volta na fila temporária, que só o solicitante consome e depois descarta. Isso simplifica a correlação (não precisa ID único, já que a resposta vai para fila dedicada), porém envolve overhead de criar e destruir filas dinamicamente.
- ❑ Trade-offs do Request-Reply: Esse padrão introduz sincronismo numa arquitetura assíncrona, portanto deve ser usado com cuidado. Ele é útil para consultas distribuídas (um serviço precisa de um dado possuído por outro agora) ou operações que requerem confirmação imediata. Contudo, deve-se gerenciar timeout: a resposta pode atrasar ou nunca chegar, então o solicitante deve ficar preparado para seguir sem ela após algum tempo. Além disso, múltiplas requisições simultâneas aumentam a complexidade e o consumo das filas de resposta.



PADRÃO REQUEST-REPLY

- ❑ Exemplo Prático: No JMS é comum usar JMSCorrelationID e JMSReplyTo para implementar request-reply. O produtor envia Message com JMSReplyTo apontando para uma TemporaryQueue, e JMSCorrelationID com um UUID. O consumidor de request faz seu trabalho e posta a resposta nessa TemporaryQueue copiando o ID. A API JMS no solicitante pode então esperar (de forma bloqueante com timeout ou assíncrona via onMessage) pela resposta nessa fila. Isso permite integrar sistemas legados que esperam resposta dentro de uma arquitetura orientada a eventos.



PREVENÇÃO DE PERDA DE DADOS EM SISTEMAS DE EVENTOS



PREVENÇÃO DE PERDA DE DADOS EM SISTEMAS DE EVENTOS

- ❑ **Desafio da Entrega Confiável:** Em comunicação assíncrona, existe o risco de perder eventos em diversos pontos: a mensagem pode não chegar ao broker, pode se perder dentro do broker, ou o consumidor pode falhar ao processar. Garantir entrega confiável é crucial para a integridade do sistema (não quereríamos perder um evento de "pagamento aprovado", por exemplo).
- ❑ **Entrega Garantida (At Least Once):** Configurar o broker e os produtores para entrega persistente. Por exemplo, utilizar filas/tópicos duráveis e marcar mensagens como persistentes no broker de mensagens. Assim, mesmo que ocorra uma falha no broker ou rede, a mensagem fica registrada em disco e será entregue assim que possível. Produtores podem usar transações ou confirmações de publicação: o produtor só considera o evento enviado após receber ack do broker que armazenou com sucesso.



PREVENÇÃO DE PERDA DE DADOS EM SISTEMAS DE EVENTOS

- ❑ Transações Producer-Broker: Uma técnica é envolver o envio de evento e a atualização local juntos numa transação. Em Java, por exemplo, o uso de JMS transacionado ou um contexto JTA permite que a inserção no banco e o envio do evento sejam atômicos: se falhar o envio, a transação aborta e não grava no banco, evitando inconsistência. Outra abordagem é o Outbox pattern: a aplicação grava o evento em uma tabela de outbox no banco dentro da transação de negócio; um processo separado lê essa tabela e envia os eventos ao broker. Isso garante que se a ação de negócio ocorreu, o evento correspondente não se perderá (será enviado posteriormente mesmo se o sistema cair logo depois).
- ❑ Confirmação de Processamento pelo Consumidor: Do lado do consumidor, usar mecanismos de acknowledgment. O consumidor só confirma/ack para o broker após ter processado e persistido o evento com sucesso. Se o consumidor falhar antes de confirmar, o broker reentrega o evento (a outro consumidor ou ao mesmo depois do reboot), prevenindo perda.



PREVENÇÃO DE PERDA DE DADOS EM SISTEMAS DE EVENTOS

- ❑ Idempotência e Duplicatas: Na busca de confiabilidade, muitas vezes opta-se por entregar pelo menos uma vez (at-least-once delivery), o que significa que duplicatas de um evento podem ocorrer (caso de reentrega após falha). É importante projetar os consumidores para serem idempotentes: repetir o processamento do mesmo evento não deve causar inconsistências (por exemplo, checar se aquela ação já foi realizada antes de aplicar novamente). Isso previne efeitos colaterais duplicados na eventualidade de mensagens reentregues.
- ❑ Monitoração e Dead Letter: Implementar filas de Dead Letter (DLQ) para onde mensagens vão após múltiplas tentativas fracassadas de entrega ou processamento. Isso evita que eventos fiquem “presos” indefinidamente tentando processar: após certo número de falhas, vão para DLQ para análise manual ou tratamento diferenciado, garantindo que nenhuma mensagem simplesmente desapareça sem rastro.



PREVENÇÃO DE PERDA DE DADOS EM SISTEMAS DE EVENTOS

- ❑ Exemplo de Perda e Prevenção: Considere que o Event Processor A envia um evento a uma fila, e o Event Processor B consome para gravar em seu banco.
- ❑ Possíveis perdas:
 - ❑ (1) A mensagem não chega à fila. Solução: A deve usar transação ou confirmação de envio;
 - ❑ (2) O broker cai antes de repassar. Solução: fila durável/persistência no broker;
 - ❑ (3) B recebe o evento mas falha antes de salvar no banco. Solução: B não confirma, permitindo reentrega após restart.
- ❑ Adotando essas práticas, garante-se resiliência: ou o evento será processado com sucesso, ou ficará registrado (em broker ou DLQ) para intervenção, mas não se perde silenciosamente.



CONCLUSÃO E RECOMENDAÇÕES

CONCLUSÃO E RECOMENDAÇÕES

- ❑ **Resumo das Topologias:** Arquitetura Orientada a Eventos fornece um modelo poderoso para sistemas distribuídos escaláveis e reativos. A Topologia Broker oferece máxima desacoplagem e throughput, brilhando em cenários de muitas fontes de evento e fluxos simples. Já a Topologia Mediator traz ordem ao caos, sendo imprescindível para workflows de múltiplas etapas com regras de negócio complexas. Cada padrão tem seu lugar: muitas implementações no mundo real combinam ambos para aproveitar vantagens de cada um.
- ❑ **Trade-off Central:** A escolha Broker vs Mediator resume-se a controle vs. escalabilidade. Se o objetivo é desempenho, simplicidade de adicionar componentes e tolerância a falhas localizadas, o Broker tende a ser preferido. Se a necessidade é confiabilidade de processo, consistência e gerenciamento de erros, o Mediator proporciona as ferramentas necessárias. Avalie os requisitos de negócio: fluxos críticos que não podem falhar em conjunto provavelmente justificam um Mediator; eventos independentes e de alto volume pedem Broker.



CONCLUSÃO E RECOMENDAÇÕES

- ❑ Padrões Complementares: Ao projetar em Java (ou qualquer stack), utilize os padrões de apoio: pub/sub para difusão de eventos; request-reply apenas quando necessário integrar consultas/retornos; sagas ou compensações para lidar com transações distribuídas em topologia Broker; idempotência e monitoramento para assegurar confiabilidade. Frameworks como Apache Camel, Spring Integration e Camel Quarkus facilitam implementar mediators e roteamentos complexos, enquanto RabbitMQ, Kafka, ActiveMQ fornecem brokers robustos para broadcast e filas. Conhecer essas ferramentas e padrões permite construir soluções event-driven resilientes.



ATÉ A PRÓXIMA AULA